

# Neural Amp Modeler A2 原理与代码导览

## Neural Amp Modeler A2 原理与代码导览

这篇笔记整理 NAM A2 的核心原理、训练代码和实时推理代码入口。A2 可以理解为 Neural Amp Modeler 的第二代默认架构配方，用来建模吉他音箱、贝斯音箱、效果器和 full rig 音色链路。

### 一句话理解

**A2 是 NAM 的新一代 WaveNet-like 架构：训练时用 PackedWaveNet 同时训练 A2-Lite 和 A2-Full，导出时生成一个 SlimmableContainer，运行时根据设备算力选择 3 通道 Lite 或 8 通道 Full。**

### A2 到底是什么

A2 不是一个单独插件，也不是某个音箱模型，而是 NAM 的新默认模型架构和训练配方。

旧架构现在被称为 **A1**。A2 则是 **Architecture 2**，目标是：

- 更高音色还原准确度
- 更低 CPU 占用
- 更适合嵌入式硬件和插件产品
- 一个模型文件同时包含 Full / Lite 两种运行尺寸
- 保持开源，方便软件、插件、硬件厂商直接集成

### A2 与 WaveNet 的关系

A2 仍然保留 WaveNet-like 的核心味道：

- 因果卷积
- dilated convolution
- residual / skip connection
- 面向音频时间序列建模

但它不是原始 WaveNet 那种自回归生成模型。原始 WaveNet 是逐 sample 预测概率分布，再生成音频；A2 是把输入音频直接映射为输出音频，用于实时效果器建模。

也就是说：

- 原始 WaveNet：生成模型
- NAM A2：实时回归模型

A2 直接输出音频 sample，并用回归损失训练。

# 架构变化

相较 A1，A2 主要变化包括：

- 激活函数从 Tanh 换成 LeakyReLU
- 使用新的卷积 head
- 在同一个 layer array 中混合不同 kernel size
- 使用重新设计的 dilation pattern
- receptive field 变大，约 6350 samples，48 kHz 下约 132 ms
- 支持 slimmable / packed training

其中 LeakyReLU 很关键。Tanh 准确但在小芯片上成本高；LeakyReLU 更便宜，省下来的 CPU 可以用来扩大网络容量，整体 accuracy-per-CPU 更好。

## A2-Lite 与 A2-Full

A2 最终保留两个尺寸：

| 模式      | 通道数        | 定位                    |
|---------|------------|-----------------------|
| A2-Lite | 3 channels | 低 CPU，适合硬件踏板、预算芯片、多实例 |
| A2-Full | 8 channels | 更高准确度，适合插件、桌面、专业音频    |

这两个尺寸在一个模型文件里，不需要分别训练。

## Slimmable NAM

slimmable 的意思是：一个训练好的模型可以用不同计算规模运行。

可以类比 IR：

- 长 IR 更精细，但 CPU 更高
- 短 IR 更省资源，但细节少一点
- 用户可以根据设备能力取舍

神经网络不能像 IR 那样直接截短，所以需要特殊训练方法。A2 用的是 packed training，它属于 slimmable NAM 的一种实现方式。

## Packed Training 的核心

Packed training 会把多个兼容的 WaveNet 子模型放进一个更大的 masked WaveNet 里一起训练。

训练时：

1. 输入同一段 dry audio
2. packed model 同时输出多个子模型的预测，例如 Lite 和 Full
3. 每个子模型预测都和同一个目标 wet audio 计算 loss
4. 所有子模型 loss 相加，反向传播
5. mask 保证不同子模型的权重块互不串扰

关键点：

- A2-Lite 和 A2-Full 共享训练流程，但不共享实际参数块
- 权重矩阵被组织成 block-diagonal 结构
- off-block 权重在 optimizer step 后被强制清零
- 导出时不是导出 packed 推理图，而是提取成多个普通 WaveNet 子模型

## 为什么不用原始 slimmable 的通道切片

早期 slimmable 思路是：小模型是大模型通道的一部分，也就是 channel sub-slice。

问题是：

- 小模型和大模型共享部分权重
- 同一组权重要同时服务两个尺寸
- 容易让两边都妥协，影响准确度

A2 的 packed variant 给每个尺寸自己的权重块。它们一起训练，但互不读写，因此 Lite 和 Full 不需要共享参数妥协。

## 训练损失

A2 训练主要使用：

- **MSE**：sample-wise 回归误差
- **MRSTFT**：multi-resolution STFT loss，用于约束频域行为
- **ESR**：主要用于 validation / checkpoint selection

TONE3000 的说明里提到，最终训练配方是经过大量 HPO 实验确定的，很多候选损失最后被舍弃，主配方保留 MSE 加小权重 MRSTFT。

## 训练代码仓库

训练和导出 `.nam` 文件的仓库：

<https://github.com/sdatkinson/neural-amp-modeler>

v0.13.0 是 A2 相关的重要版本：

<https://github.com/sdatkinson/neural-amp-modeler/releases/tag/v0.13.0>

这个 release 包含：

- 新 activations
- WaveNet bottlenecks
- WaveNet head 1x1 layer
- slimmable WaveNet
- Packed WaveNet training
- 简化 trainer 默认使用 A2

## 训练端关键源码

### A2 默认配置

文件：

[https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/train/\\_resources/config\\_model\\_packed.json](https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/train/_resources/config_model_packed.json)

这里定义了 A2 的默认 packed model：

- name: PackedWaveNet
- 子模型 channels\_3
- 子模型 channels\_8
- activation: LeakyReLU
- head scale: 0.01
- loss: ESR validation + MRSTFT 权重
- optimizer learning rate 等训练参数

这个文件是理解 A2 具体结构最直接的入口。

### PackedWaveNet 配置构造

文件：

[https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/\\_packed.py](https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/_packed.py)

作用：

- 验证多个子模型是否结构兼容
- 要求 kernel size、dilation、activation 等时间结构一致
- 允许 channel count 不同
- 把多个子模型 config 拼成一个 packed WaveNet config

## Masked Conv

文件：

[https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/\\_packed\\_conv.py](https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/_packed_conv.py)

这是 packed training 的核心之一。

`PackedConv1dBase` 会创建一个和权重同形状的 mask：

- 允许当前子模型的 output block 读取自己的 input block
- 禁止读取其他子模型的 channel block
- forward 时使用 `weight * mask`
- optimizer 更新后调用 `apply_mask()`，把 off-block 权重重新清零

这就是 A2 packed training 能让 Lite / Full 一起训练但参数互不干扰的关键。

## PackedWaveNet 类

文件：

[https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/\\_packed\\_wavenet.py](https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/models/wavenet/_packed_wavenet.py)

作用：

- 包装内部 WaveNet
- forward 输出形状类似 `(batch, submodel, time)`
- `extract_submodel()` 可以从 packed model 中提取单个普通 WaveNet
- `export_container()` 会导出 `SlimmableContainer`

导出的 `.nam` 文件不是 packed 运行图，而是：

```
SlimmableContainer
├── submodel: A2-Lite / channels_3 / max_value ...
└── submodel: A2-Full / channels_8 / max_value 1.0
```

## PackedLightningModule

文件：

[https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/train/lightning\\_module.py](https://github.com/sdatkinson/neural-amp-modeler/blob/main/nam/train/lightning_module.py)

重点类：

- `PackedLightningModule`
- `PackedMaskCallback`
- `PackedBestCheckpoint`

训练逻辑：

- 对每个 packed 子模型分别计算 loss
- 所有 loss 相加作为总 loss
- validation 时记录每个子模型的 ESR / MRSTFT / val\_loss
- 可以为每个子模型保存自己的 best checkpoint

## 官方 packed training 文档

文档：

<https://neural-amp-modeler.readthedocs.io/en/latest/tutorials/packed-training.html>

几个重要结论：

- packed training 用普通 `nam-full` 命令，不需要额外 flag
- 配置里写 `"PackedWaveNet"` 即可
- 子模型必须结构兼容
- channel count 可以不同
- 输出 `model.nam` 的 architecture 是 `SlimmableContainer`
- packed training model 本身不是 runtime format

## 实时推理代码仓库

实时 DSP / 插件 / 产品集成使用的 C++ Core 仓库：

<https://github.com/sdatkinson/NeuralAmpModelerCore>

A2 支持相关版本：

<https://github.com/sdatkinson/NeuralAmpModelerCore/releases/tag/v0.5.3>

软件或插件要支持 A2，需要升级到支持 A2 的 NeuralAmpModelerCore。Core 中默认开启 `NAM_ENABLE_A2_FAST`，匹配 A2 形状模型会走专门优化过的 fast path。

## 推理端关键源码

### Slimmable 接口

文件：

<https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/slimmable.h>

核心接口：

```
class SlimmableModel
{
public:
    virtual void SetSlimmableSize(const double val) = 0;
};
```

`SetSlimmableSize()` 用于设置模型运行尺寸。参数通常是 0.0 到 1.0，具体解释由模型实现决定。

## SlimmableContainer

文件：

<https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/container.cpp>

作用：

- 读取 `.nam` 文件里的 `SlimmableContainer`
- 内部持有多个 submodel
- 默认使用最后一个 submodel，也就是最高质量模型
- `SetSlimmableSize()` 根据 `max_value` 阈值切换 active submodel
- `process()` 只调用当前 active submodel

也就是说，运行时并不是同时跑 Full 和 Lite，而是只跑当前选择的那一个。

## A2 fast path

文件：

[https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/wavenet/a2\\_fast.h](https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/wavenet/a2_fast.h)

[https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/wavenet/a2\\_fast.cpp](https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/wavenet/a2_fast.cpp)

`a2_fast` 是 A2 专门优化推理路径。它会严格检测 WaveNet config 是否匹配 A2 形状。

匹配条件包括：

- layer 数量是 23
- channel count 是 3 或 8
- kernel size pattern 固定
- dilation pattern 固定

- activation 是 LeakyReLU
- head kernel size 是 16
- 没有 FiLM
- 没有 grouped conv
- 不是 channel-slicing slimmable WaveNet

如果匹配，就实例化 `A2FastModel<3>` 或 `A2FastModel<8>`。

## WaveNet config parser

文件：

<https://github.com/sdatkinson/NeuralAmpModelerCore/blob/main/NAM/wavenet/model.cpp>

逻辑大致是：

1. 如果 config 是 slimmable WaveNet，走 slimmable WaveNet
2. 如果开启 `NAM_ENABLE_A2_FAST` 且 config 匹配 A2 shape，走 A2 fast path
3. 否则走普通 WaveNet

这说明 A2 的运行支持不是靠“文件名里有 A2”，而是靠模型 config 的结构签名来判断。

## A2 模型文件的大致结构

A2 `.nam` 文件大致会是：

```
{
  "architecture": "SlimmableContainer",
  "config": {
    "submodels": [
      {
        "max_value": 0.5,
        "model": {
          "architecture": "WaveNet",
          "config": {
            "channels": 3
          }
        }
      },
      {
        "max_value": 1.0,
        "model": {
          "architecture": "WaveNet",
          "config": {
            "channels": 8
          }
        }
      }
    ]
  }
}
```



```
}  
  }  
  }  
  ]  
},  
  "weights": []  
}
```

实际文件会更复杂，包含完整 layer config、weights、metadata、sample rate 等。

## 阅读源码建议顺序

如果要真正读懂 A2，可以按这个顺序：

1. 先读 A2 默认配置  
`nam/train/_resources/config_model_packed.json`
2. 再读 packed training 文档  
`docs/source/tutorials/packed-training.rst`
3. 看 packed config 如何构造  
`nam/models/wavenet/_packed.py`
4. 看 masked conv 如何隔离子模型  
`nam/models/wavenet/_packed_conv.py`
5. 看 PackedWaveNet 如何导出 container  
`nam/models/wavenet/_packed_wavenet.py`
6. 看训练时如何计算多个子模型 loss  
`nam/train/lightning_module.py`
7. 看推理端 container 如何切换模型  
`NeuralAmpModelerCore/NAM/container.cpp`
8. 最后看 A2 fast path 的低层优化  
`NeuralAmpModelerCore/NAM/wavenet/a2_fast.cpp`

## 相关链接

- A2 视频: <https://www.youtube.com/watch?v=MtXtthiEH18>
- A2 完整指南: <https://www.tone3000.com/guides/nam-a2-the-complete-guide>
- A2 发布博客: <https://www.neuralampmodeler.com/post/a2-is-released>
- A2 早期架构说明: <https://www.neuralampmodeler.com/post/architecture-a2>
- Slimmable NAM 论文: <https://arxiv.org/abs/2511.07470>
- 训练代码: <https://github.com/sdatkinson/neural-amp-modeler>
- 训练代码 v0.13.0: <https://github.com/sdatkinson/neural-amp-modeler/releases/tag/v0.13.0>
- Packed training 文档: <https://neural-amp-modeler.readthedocs.io/en/latest/tutorials/packed-training.html>

- 推理 Core: <https://github.com/sdatkinson/NeuralAmpModelerCore>
- 推理 Core v0.5.3:  
<https://github.com/sdatkinson/NeuralAmpModelerCore/releases/tag/v0.5.3>

## 小结

A2 的关键不是“一个更大的模型”，而是一次工程化很强的架构升级：

- 训练上，用 packed training 同时得到 Lite 和 Full
- 文件格式上，用 SlimmableContainer 把多个子模型装进一个 `.nam`
- 推理上，用 container 选择当前子模型
- 性能上，用 A2 fast path 对固定形状做专门优化
- 产品化上，让低成本硬件有机会直接原生运行 NAM，而不是把 NAM 转换成别的简化模型

这也是 A2 对 NAM 生态最重要的意义：它把高质量 neural amp modeling 从桌面插件进一步推向硬件设备和开放产品生态。